FIN-RSI: An Agent-Native Library of Financial Skills, Checks, and Self-Improvement

Anonymous submission

Abstract

Financial research code fails silently: a program that applies stock splits twice, or trades on the closing price it reads, still runs and still returns a backtest. We present FIN-RSI, an agent-native Python library that puts three things behind interfaces a language-model agent can call: *skills* (131 dated skill documents, 48 algorithms and 6 rule cards), *checks* (39 guards, including property checks that need no expected output) and *self-improvement* (7 methods behind one `fit`/`solve`/`state` interface with metered, equal budgets). With it comes a suite of 34 financial programming tasks on synthetic and real prices, each with an independent reference and a hidden grader that no method sees. A study fixed in advance runs 20 open-weight models through controlled feedback and library arms, the 7 methods and 367 public BizBench questions. [One sentence of results.] The library, tasks and protocol will be released.

1 Introduction

Financial research code fails silently. A program that applies stock splits twice, decides a position with the close it trades over, or leaves warm-up rows missing still runs and still returns a backtest (Figure 2). Language models are moving into financial work (Nie et al. 2024; Kong et al. 2024; Dong et al. 2025), and agents now write such code (Zhang et al. 2024; Bigeard et al. 2025). They make these mistakes: in our earlier audit, all 24 programs a 14B coding model wrote re-applied split factors that the prompt defined as already cumulative. The problem does not go away when a foundation model supplies the forecast or the portfolio weights (Jin et al. 2024; Liang et al. 2024; Hwang et al. 2025): the code that turns them into positions must still apply each split once and trade only on what was known.

An agent would need three things to avoid them: knowledge of the failure modes, checks that detect each failure without an expected output, and a way to turn check results into better programs. All three exist, but separately. Skill documents and tool collections give knowledge; testing utilities give checks; self-improvement methods each come with their own tasks, feedback and budgets (Shinn et al. 2023; Chen et al. 2024; Zhao et al. 2024; Agrawal et al. 2026), so their results cannot be compared. Intrinsic self-correction without an external signal rarely helps (Huang et al. 2024), and financial benchmarks score answers or returns (Xie et al. 2024; Chen et al. 2025b); a return can reward a wrong program on a favourable price path, and repeated trials make such false discoveries likely (Bailey et al. 2017). Evaluation practice has the same gap: of 164 recent papers on language models in finance, about a quarter acknowledge look-ahead bias (Kong et al. 2026).

FIN-RSI puts all three behind interfaces an agent can call (Figure 1). PyOD put outlier detectors behind one interface (Zhao, Nasrullah, and Li 2019; Chen et al. 2025a), which let ADBench compare them on equal terms (Han et al. 2022). FIN-RSI does the same for self-improvement: every method gets the same judge and the same metered budget. A user connects a model once, as one `generate(messages)` callable. Our contributions:

- an agent-native library of skills, checks and tools (§2);
- 7 self-improvement methods behind one interface with equal budgets (§3);
- 34 financial programming tasks with independent references, a hidden grader, an algorithm-level held-out split and real-price panels (§4);
- a study fixed before any test run, on 20 open-weight models: feedback against plain resampling, the library against none, the 7 methods, and a check on public BizBench questions (§5).

2 The FIN-RSI library

FIN-RSI is a Python package whose base install needs only numpy, pandas and scipy.

Skills. 131 skill documents in 17 plugins follow the Agent Skills specification (Agent Skills 2026), with 75 reference files and 125 runnable scripts. A skill records the defaults, licence changes and silent failure modes that a model's training prior gets wrong; each claim is dated and marked by how it was verified. A further 92 federated packs (2,390 skills, off by default, not verified here) include 100 investor templates built on named investors' approaches. A catalog of 48 algorithms covers 11 task types (25 native, 23 adapters to 13 optional libraries, reported as missing rather than replaced

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

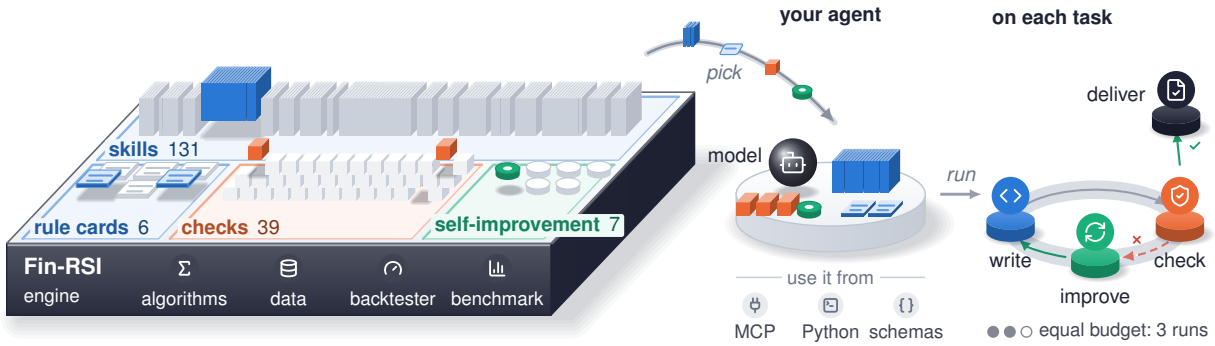


Figure 1: What is in Fin-RSI and how an agent uses it. Left: the library as a cabinet whose tray holds one piece per item, in the real counts (a book per skill, grouped by plugin; the rule cards; a cube per check; a token per self-improvement method), with the engine on its front. Middle: an agent is the user's own model plus the pieces it picks, which rise from the tray; it is reached through MCP, tool schemas or the Python API. Right: on each task the agent writes a program and the checks run; a failed check goes to the self-improvement method before the next run, within one budget for every method, and a program that passes is delivered. Figure 2 follows one episode.

when absent). `positions()` takes raw quotes and split factors, applies the factors once and returns positions decided on the previous close; `rolling()` does the same for volatility, risk and forecasts. Between documents and code sit 6 rule cards in 3 families (timing, corporate actions, data hygiene; Appendix Table 2). A card holds a short rule, the skill passage it comes from, the checks that verify it and, where one exists, the library call that follows it. A task declares traits such as *split factors* and receives only the cards they select.

Checks. The library has 39 guards behind one interface, which returns whether the guard passed, a message and warnings; 32 take JSON inputs and are exported as tools. For generated programs, three property checks need no expected output:

- `finite_output`: no missing or infinite values;
- `split_representation`: re-run on the same economic prices, given as adjusted closes with unit factors, the output must not change (10^{-8});
- `execution_lag`: perturb prices from row k on; no position up to row k may move (Figure 2c). A rule that trades on the close it reads fails, although it passes a plain causality test.

These are metamorphic relations (Segura et al. 2016) for financial code. Each check carries a *cause*, one sentence on what it observed, and `explain()` pairs it with the cards that address it. We count the cause as part of the check and the card as part of the skills.

Agent interfaces. One registry yields 67 tools under the same names as a Model Context Protocol server (Model Context Protocol 2026) and as OpenAI and Anthropic tool schemas: read and search skills, run algorithms, run guards. `agents.Team` turns the user's `generate` callable into a writer and one reviewer per rule family; only the checks decide acceptance.

3 Self-improvement behind one interface

Every method is a class with one constructor and three calls:

```
m = rsi.Reflexion(library=True)
m.fit(generate, tasks.dev)
ep = m.solve(generate, task, seed=11)
ep.accepted, ep.code, ep.usage
m.state()
```

`state()` returns what the method learned and is hashed into every episode record; `fit()` does nothing for a method that does not learn. `rsi.benchmark()` fits each method once on the development tasks and runs it on every held-out task and seed.

What a method sees. `solve()` gets the task text and a judge on which it can only call `check(code)`. That returns whether the program ran, its error, each check's result and the causes of the failed ones, never the grade (Figure 2b). Learning methods may use development grades to keep or reject a candidate, but no text a model reads contains a grade.

Equal budgets. A meter refuses any call beyond the budget and records which limit stopped the episode. Each episode gets 3 program executions, 6 model calls (reflections and reviews included) and 12,000 generated tokens. Learning methods also get 48 development episodes, 6 proposals, 600 calls and 1,200,000 tokens.

Methods. The 7 methods (Appendix Table 1) keep the mechanism each is known for: `NoImprove` (one program); `SelfDebug`, which returns only errors (Chen et al. 2024); `CheckFeedback`, which returns the failed checks with causes and cards; `Reflexion`, which adds a written reflection before each retry (Shinn et al. 2023); `Team`, with one reviewer per rule family; `Evolve`, which evolves a prompt addendum on development tasks (Agrawal et al. 2026); and `PlaybookLearning` (Zhao et al. 2024; Suzgun et al. 2026). The playbook proposer sees development traces without grades, and one proposal edits one entry. A candidate is accepted only if its score does not drop on either of two

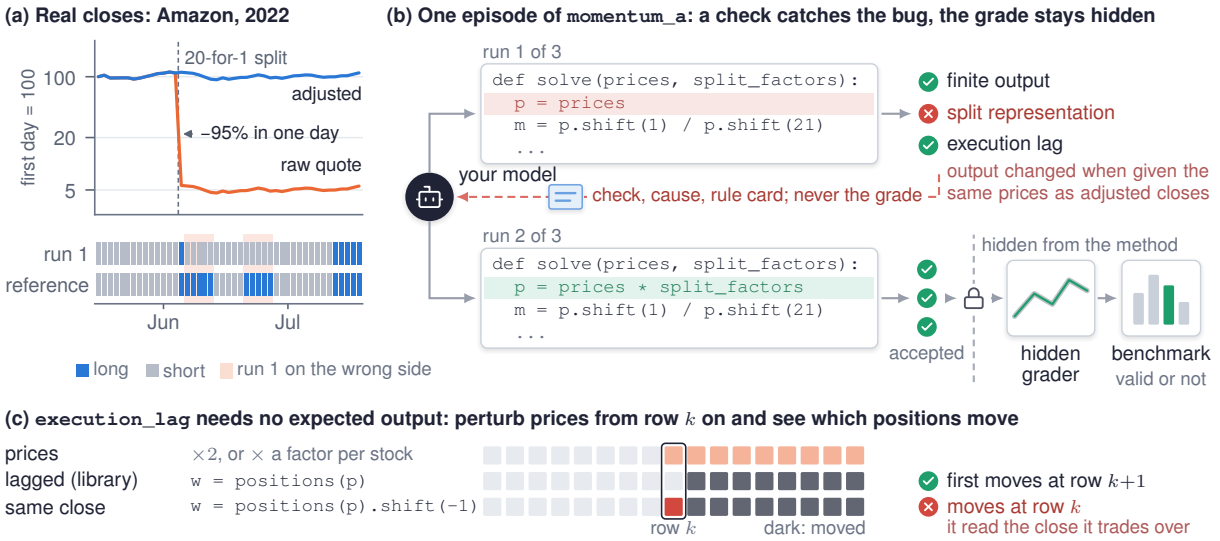


Figure 2: One episode: a one-line bug that runs without error, caught by a check and confirmed by a grader the method never sees. (a) Amazon's 20-for-1 split on 6 June 2022 (real closes, indexed): raw quotes fall 95% in one day. Run 1 of (b) reads the raw quotes, so its momentum signal is short Amazon for the 25 sessions after the split; on 10 of them (shaded) the task's independent reference is long. (b) On task `momentum_a` the judge runs three checks. Run 1 fails `split_representation`; the method gets the failed check, its cause (shortened here) and the `split_once` rule card, never the grade, and retries within its budget of 3 runs. Run 2 changes that one line and passes. Only then, behind the wall, does the hidden grader compare the final program with the reference; its verdict goes to the benchmark alone. These are the judge's real verdicts on every synthetic seed and on the real panel. (c) The third check on a synthetic panel: the library's lagged inverse-volatility positions first move at row $k{+}1$ and pass; read one row early, they move at row k and fail.

disjoint halves of the development tasks, rises on one, and no task that passed every check before now fails one (the *no-regression* rule; Appendix Figure 3).

Controlled arms. A retry can help because of what the judge returned or simply because the model sampled again. Three arms separate the two. They share one prompt and one budget and differ only in what follows a failed attempt: `Repeat` returns nothing and samples again; `Check` returns the error, or the names of the failed checks; `Explain` adds the cause of each failed check. Like every method, each arm runs with and without the `library` switch, which adds the interface text to the prompt and allows `import fin_skills`. No first prompt contains a rule card, and every arm delivers the first program that passes the visible checks.

4 Task suite

34 tasks cover 17 algorithms in 5 families, two variants each (Appendix Table 3). A task states a rule in prose; the model implements `solve(prices, split_factors)` returning positions, or for volatility, risk and forecast tasks the value known at the previous close. Each task can be solved by hand or by one library call. The rules are the momentum, volatility-scaling and allocation baselines that learned strategies are measured against (Lim, Zohren, and Roberts 2019; Wood, Roberts, and Zohren 2022; Zhang, Zohren, and Roberts 2020).

Data. Synthetic panels have four assets and 128 business days of seeded Gaussian returns, with four splits at fixed rows (2-for-1, 3-for-1, 3-for-2, 1-for-2), on 3 seeds. Variant a gives cumulative factors in alphabetical order; variant b gives event ratios, shuffled columns and other parameters. The suite also has a real panel: as-traded closes of Amazon, Shopify, Alphabet and Tesla for 128 sessions from 31 March 2022 to 3 October 2022, in which all four split (20-, 10-, 20- and 3-for-1; Appendix Figure 4). Yahoo Finance serves the closes split-adjusted; we rebuild as-traded quotes from a split table checked against the companies' announcements and published pre-split closes, and release the fetch code and the file's SHA-256, since the prices may not be redistributed.

Judge and hidden grader. The judge runs each program in a separate process with a memory limit and a 120-second timeout (in our runs also confined with Landlock and seccomp), refuses `import fin_skills` without library access, and applies the three checks of §2. A program is *valid* only if it ran, passed every check and matches an independent numpy reference within 10^{-8} (10^{-5} for `min_variance`). A program that passes the checks but is wrong is *accepted but wrong*; we report that rate because the checks cannot see it.

Qualification. Before any model ran: on every task and data panel, the reference agrees with the library to within 2.8×10^{-14} (1.4×10^{-6} for `min_variance`); a correct program passes every check and is graded valid; and on a position task and a table task, planted versions of each

failure (same close, no adjustment, factors cumulated twice, missing warm-up) fail their check and the grade. Building the references exposed a library defect: the long-only minimum-variance optimiser stopped early on daily-scale covariances, off by up to 1.2×10^{-2}, until we rescaled the covariance before solving.

Split. The split is by algorithm. The 5 algorithms a model has already seen go to development, and a seeded draw sends 4 of the other 12 there too: 18 development and 16 held-out tasks, the latter covering all 5 families.

5 Study protocol

We fixed the tasks, prompts, seeds, budgets, models and contrasts before any test run, on the suite as it stood on 8 November 2026: 12 development and 6 held-out position tasks on synthetic panels (Bollinger, RSI and Donchian rules, two variants each). The tasks added since, and the real panel, are not part of the study. It has three tracks.

P: feedback and library. The three controlled arms, with and without the library, on the 6 held-out tasks and 3 seeds: 108 episodes per model. Three contrasts were declared: `Check` against `Repeat` and `Explain` against `Check`, both without the library, and library against none under `Explain`.

E: external questions. 367 public questions from the two table-and-text program-synthesis tasks of BizBench (Krumdick et al. 2024) (200 CodeFinQA, 167 CodeTAT-QA), selected by a rule fixed before any model output: a numeric answer, a prompt that fits every core model's context, no duplicate in the training split, one question per group of shared context. The model writes one program per question, with and without the library and without feedback: 2,202 episodes per model. An answer counts if the program runs and returns the public label exactly; BizBench scores within 1%, which we report as a sensitivity analysis.

S: seven methods. The 7 methods of §3, with and without the library, on the same tasks and seeds, for 5 core models: 1,260 episodes and 20 fits on the development tasks.

Models and analysis. Tracks P and E run on 20 open-weight instruction models from 8 families, 2B to 32B parameters, in BF16 at temperature 0.1, with a 32,768-token context (16,384 for one model with a smaller native limit) and at most 2,048 generated tokens per call. Every refusal, truncated reply and failed program stays in the denominator. The position tasks cover only 3 algorithms, and variants and seeds are not independent of them: with three clusters a sign-flip test cannot go below $p = 0.25$. We therefore report P and S as effects per algorithm, without significance claims. Track E weights its two sources equally.

6 Preliminary results

[Per track and model, for the models whose runs finished: valid rate by arm (P), by library condition (E) and by method (S), each with the accepted-but-wrong rate, the share of replies with no program, and the cost in calls and tokens; the three P contrasts per algorithm. Models still running are listed as incomplete, not averaged in.]

Limitations. The study covers three position algorithms on synthetic panels and one set of public questions; the newer tasks, the real panel and the rule-card switch are released but not evaluated here. All tasks are public, so a model may have seen them. The checks reject only the failures they were written for, hence the accepted-but-wrong rate. In the frozen source, `Evolve` chose the development failures it showed its proposer by the hidden grade; the release fixes this, and the `Evolve` runs reported here predate the fix. All models are open-weight, under one attempt budget.

References

Agent Skills. 2026. Agent Skills Specification. https://agentskills.io/specification. Accessed 2026-11-04.

Agrawal, L. A.; Tan, S.; Soylu, D.; Ziems, N.; Khare, R.; Opsahl-Ong, K.; Singhvi, A.; Shandilya, H.; Ryan, M. J.; Jiang, M.; Potts, C.; Sen, K.; et al. 2026. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.

Bailey, D. H.; Borwein, J.; López de Prado, M.; and Zhu, Q. J. 2017. The Probability of Backtest Overfitting. *Journal of Computational Finance*, 20(4): 39–69.

Bigeard, A.; Nashold, L.; Krishnan, R.; and Wu, S. 2025. Finance Agent Benchmark: Benchmarking LLMs on Real-world Financial Research Tasks. *arXiv preprint arXiv:2508.00828*.

Chen, S.; Qian, Z.; Siu, W.; Hu, X.; Li, J.; Li, S.; Qin, Y.; Yang, T.; Xiao, Z.; Ye, W.; Zhang, Y.; Dong, Y.; and Zhao, Y. 2025a. PyOD 2: A Python Library for Outlier Detection with LLM-powered Model Selection. In *Companion Proceedings of the ACM on Web Conference 2025*, 2807–2810.

Chen, X.; Lin, M.; Schärli, N.; and Zhou, D. 2024. Teaching Large Language Models to Self-Debug. In *International Conference on Learning Representations (ICLR)*.

Chen, Y.; Yao, Z.; Liu, Y.; Xin, A.; Ye, J.; Yu, J.; Hou, L.; and Li, J. 2025b. StockBench: Can LLM Agents Trade Stocks Profitably In Real-world Markets? *arXiv preprint arXiv:2510.02209*.

Dong, Y.; Wu, F.; Zhang, K.; Dai, Y.; Zhang, S.; Ye, W.; Chen, S.; and Cheng, Z.-Q. 2025. Large Language Model Agents in Finance: A Survey Bridging Research, Practice, and Real-World Deployment. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 17889–17907.

Han, S.; Hu, X.; Huang, H.; Jiang, M.; and Zhao, Y. 2022. ADBench: Anomaly Detection Benchmark. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*.

Huang, J.; Chen, X.; Mishra, S.; Zheng, H. S.; Yu, A. W.; Song, X.; and Zhou, D. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *International Conference on Learning Representations (ICLR)*.

Hwang, Y.; Kong, Y.; Zohren, S.; and Lee, Y. 2025. Decision-informed Neural Networks with Large Language

Model Integration for Portfolio Optimization. *arXiv preprint arXiv:2502.00828*. Journal version: Expert Systems with Applications 334, 134389, doi:10.1016/j.eswa.2026.134389.

Jin, M.; Wang, S.; Ma, L.; Chu, Z.; Zhang, J. Y.; Shi, X.; Chen, P.-Y.; Liang, Y.; Li, Y.-F.; Pan, S.; and Wen, Q. 2024. Time-LLM: Time Series Forecasting by Reprogramming Large Language Models. In *International Conference on Learning Representations (ICLR)*.

Kong, Y.; Lee, H.; Hwang, Y.; Lopez-Lira, A.; Levy, B.; Mehta, D.; Wen, Q.; Choi, C.; Lee, Y.; and Zohren, S. 2026. Evaluating LLMs in Finance Requires Explicit Bias Consideration. *arXiv preprint arXiv:2602.14233*.

Kong, Y.; Nie, Y.; Dong, X.; Mulvey, J. M.; Poor, H. V.; Wen, Q.; and Zohren, S. 2024. Large Language Models for Financial and Investment Management: Applications and Benchmarks. *The Journal of Portfolio Management*, 51(2): 162–210.

Krumdick, M.; Koncel-Kedziorski, R.; Lai, V. D.; Reddy, V.; Lovering, C.; and Tanner, C. 2024. BizBench: A Quantitative Reasoning Benchmark for Business and Finance. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 8309–8332.

Liang, Y.; Wen, H.; Nie, Y.; Jiang, Y.; Jin, M.; Song, D.; Pan, S.; and Wen, Q. 2024. Foundation Models for Time Series Analysis: A Tutorial and Survey. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*, 6555–6565.

Lim, B.; Zohren, S.; and Roberts, S. 2019. Enhancing Time-Series Momentum Strategies Using Deep Neural Networks. *The Journal of Financial Data Science*, 1(4): 19–38.

Model Context Protocol. 2026. Model Context Protocol Specification, revision 2026-07-28. https://modelcontextprotocol.io/specification.

Nie, Y.; Kong, Y.; Dong, X.; Mulvey, J. M.; Poor, H. V.; Wen, Q.; and Zohren, S. 2024. A Survey of Large Language Models for Financial Applications: Progress, Prospects and Challenges. *arXiv preprint arXiv:2406.11903*.

Segura, S.; Fraser, G.; Sánchez, A. B.; and Ruiz-Cortés, A. 2016. A Survey on Metamorphic Testing. *IEEE Transactions on Software Engineering*, 42(9): 805–824.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Suzgun, M.; Yuksekgonul, M.; Bianchi, F.; Jurafsky, D.; and Zou, J. 2026. Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, 7080–7106.

Wood, K.; Roberts, S.; and Zohren, S. 2022. Slow Momentum with Fast Reversion: A Trading Strategy Using Deep Learning and Changepoint Detection. *The Journal of Financial Data Science*, 4(1): 111–129.

Xie, Q.; Han, W.; Chen, Z.; Xiang, R.; Zhang, X.; He, Y.; Xiao, M.; Li, D.; Dai, Y.; Feng, D.; Xu, Y.; et al. 2024. FinBen: A Holistic Financial Benchmark for Large Language Models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 95716–95743.

Zhang, W.; Zhao, L.; Xia, H.; Sun, S.; Sun, J.; Qin, M.; Li, X.; Zhao, Y.; Zhao, Y.; Cai, X.; Zheng, L.; Wang, X.; and An, B. 2024. A Multimodal Foundation Agent for Financial Trading: Tool-Augmented, Diversified, and Generalist. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*, 4314–4325.

Zhang, Z.; Zohren, S.; and Roberts, S. 2020. Deep Learning for Portfolio Optimization. *The Journal of Financial Data Science*, 2(4): 8–20.

Zhao, A.; Huang, D.; Xu, Q.; Lin, M.; Liu, Y.-J.; and Huang, G. 2024. ExpeL: LLM Agents Are Experiential Learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 19632–19642.

Zhao, Y.; Nasrullah, Z.; and Li, Z. 2019. PyOD: A Python Toolbox for Scalable Outlier Detection. *Journal of Machine Learning Research (JMLR MLOSS)*, 20(96): 1–7.

A Self-improvement methods

`Reflexion` asks for a reflection of at most five sentences, without code, and keeps the reflections for the rest of the episode. In `Team`, a reviewer is asked only when a check of its own family failed; it sees the program, the failed checks, their causes and its family's cards, and answers with a diagnosis rather than code. `Evolve` rewrites a prompt addendum of at most 1,200 characters from the visible failures of the current best addendum, and keeps a candidate only if it scores strictly higher on the same development tasks. The playbook is a list of short entries on three surfaces (tool usage, retrieval, experience). Its loop rejects whitespace-only and repeated edits, and appends every proposal, evaluation and decision to a journal whose receipts carry evidence hashes; a frozen playbook cannot change. A stricter *all-pass* rule, which vetoes any candidate while a single task fails, blocks learning for a weak model, so we use no-regression.

B Real-price panel

C Rule cards

D Task suite

Method	At test time	Learned on development tasks	In the spirit of
`NoImprove`	one program, no feedback	–	baseline
`SelfDebug`	retry with the error only; stop once the program runs	–	Chen et al. (2024)
`CheckFeedback`	retry with the failed checks, their causes and cards	–	verifier feedback
`Reflexion`	as `CheckFeedback`, plus a written reflection before each retry	–	Shinn et al. (2023)
`PlaybookLearning`	as `CheckFeedback`, with a frozen playbook in the prompt	playbook entries, through the gated loop	Zhao et al. (2024); Suzgun et al. (2026)
`Team`	a writer and one reviewer per rule family, from the same model	–	multi-role review
`Evolve`	as `CheckFeedback`, with an evolved prompt addendum	the addendum, over 3 generations on 4 tasks	Agrawal et al. (2026)

Table 1: The 7 self-improvement methods. All share the prompt, the judge and the budget; each keeps the mechanism it is known for. They are not reproductions of the original systems.

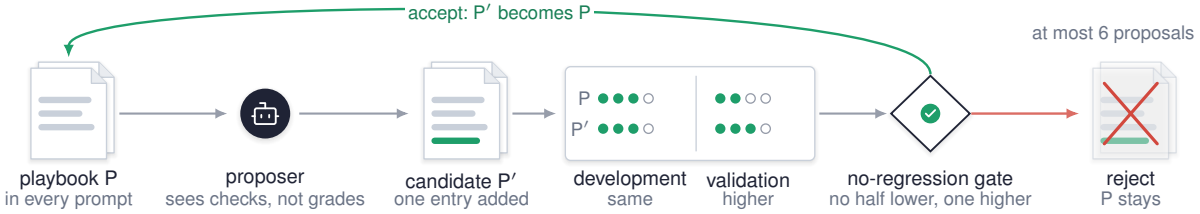


Figure 3: The gated playbook loop of `PlaybookLearning`. The proposer reads development traces without grades and adds one entry; the current and the candidate playbook are scored on two disjoint halves of the development tasks (dots: tasks that passed their checks; an example, not results). The no-regression gate accepts the candidate only if neither half is lower, one is higher and no task that passed before now fails.

Card	Verified by	Call
timing		
`execution_lag`	`execution_lag`	yes
`feature_causality`	`assert_causal`	–
corporate actions		
`split_once`	`split_representation` `adjustment_check`	yes
`same_convention`	`split_representation`	–
data hygiene		
`warmup`	`finite_output`	–
`first_return`	`finite_output`	–

Table 2: Rule cards by family, the checks that verify each one, and whether the card names a library call that follows the rule.

Algorithm	Split	Library call
portfolio		
`cross_sectional_momentum`	dev	`positions()`
`hrp`	heldout	`positions()`
`inverse_volatility`	dev	`positions()`
`min_variance`	heldout	`positions()`
signal		
`bollinger_reversion`	dev	`positions()`
`donchian_breakout`	heldout	`positions()`
`ma_crossover`	dev	`positions()`
`macd`	dev	`positions()`
`momentum`	dev	`positions()`
`rsi_reversion`	heldout	`positions()`
`vol_target_momentum`	dev	`positions()`
volatility		
`ewma_volatility`	heldout	`rolling()`
`historical_volatility`	dev	`rolling()`
risk		
`historical_var_es`	heldout	`rolling()`
`normal_var_es`	heldout	`rolling()`
forecast		
`drift`	heldout	`rolling()`
`mean`	dev	`rolling()`

Table 3: The 17 algorithms of the task suite, with their split and the one-step library call that solves them. Each algorithm has two tasks (variants a and b).

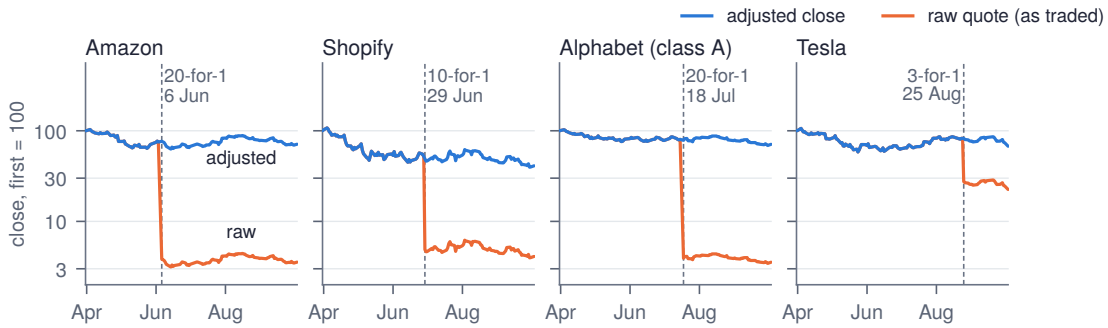


Figure 4: The real-price panel: raw quotes (as traded) and adjusted closes of the four stocks, each indexed to 100 at the first close, on a log scale. Every stock splits inside the window; a program that ignores the split factors sees a one-day loss of 67% to 95%.